# Technical Report on the Notebook

Mango Detection and Ripeness Classification

Prepared from the saved project notebook

23 April 2026

## Contents


**1 Abstract** **3**

**2 Project Context and Problem Statement** **3**
- 2.1 What problem the notebook solves . . . 3
- 2.2 Why this problem matters . . . 4

**3 Primary Source of This Report** **4**

**4 Notebook Audit Summary** **4**
- 4.1 Structure . . . 4
- 4.2 Execution state . . . 4
- 4.3 Section-by-section audit . . . 4

**5 System Objective and Functional Scope** **5**

**6 Technical Architecture** **6**
- 6.1 Two-stage pipeline . . . 6
- 6.2 Why this architecture is appropriate . . . 6
- 6.3 Confidence logic . . . 6

**7 Detailed Implementation Analysis** **7**
- 7.1 Imported libraries . . . 7
- 7.2 Global settings and constants . . . 7
- 7.3 Inference utility functions . . . 8

**8 Datasets and Data Sources** **9**
- 8.1 Detection dataset . . . 9
- 8.2 Ripeness classification dataset . . . 9
- 8.3 Why two datasets are used . . . 9

**9 Training Configuration** **10**
9.1 YOLOv8 detector . . . 10
9.2 EfficientNetB0 classifier . . . 10
9.3 Data loading for the classifier . . . 11

**10 Execution Evidence from the Saved Notebook** **11**
10.1 Observed completed stages . . . 11
10.2 Observed detector evidence . . . 11
10.3 Observed incomplete stages . . . 12

**11 Step-by-Step Operational Flow** **12**

**12 The Streamlit Deployment Layer** **13**

**13 Design Strengths** **13**

**14 Limitations and Risks** **14**
14.1 Partial execution record . . . 14
14.2 Hard-coded secret . . . 14
14.3 Cross-dataset domain gap . . . 14
14.4 Unclear label alignment . . . 14
14.5 Class imbalance risk . . . 14

**15 Improvement Plan Documented by the Notebook** **14**

**16 Recommendations for a More Complete Final Project** **15**

**17 Conclusion** **15**

**18 References** **16**

# 1 Abstract

This report documents the saved Jupyter notebook `advance_mango_detection.ipynb`, which implements a two-stage computer vision pipeline for mango analysis. The notebook is designed to solve two related problems: first, detecting mangoes in an image; second, classifying the ripeness state of each detected mango as `raw`, `ripe`, or `overripe`. The implementation combines a YOLOv8 object detector for localization with an EfficientNetB0-based transfer-learning classifier for ripeness recognition [2, 6, 7, 8]. The notebook also adds practical safeguards such as confidence thresholds, margin-based cropping, uncertain-label handling, no-mango detection logic, model export, and a Streamlit deployment script [10]. This report explains what problem the notebook solves, why the problem matters, how the solution is built, what each notebook stage does, what data sources and libraries are used, what was actually executed in the saved notebook, and what limitations remain.

# 2 Project Context and Problem Statement

## 2.1 What problem the notebook solves

The notebook addresses an agricultural and quality-assessment problem: automatically identifying mangoes in images and estimating their ripeness stage. This is useful in settings such as:

- post-harvest sorting,
- warehouse quality control,
- retail inspection,
- crop monitoring and decision support,
- educational demonstrations of end-to-end computer vision pipelines.

The notebook does not treat the task as a single flat image-classification problem. Instead, it separates the workflow into:

1. **detection**: find where mangoes are in the image;
2. **classification**: classify the ripeness of each detected fruit.

This design is important because a single image may contain multiple mangoes, no mangoes at all, or mangoes at different ripeness stages. A detection-first pipeline is therefore more realistic than directly classifying the whole image.

## 2.2 Why this problem matters

Ripeness estimation influences harvesting, transport timing, market pricing, and waste reduction. A robust automated system can improve consistency and reduce human subjectivity. The notebook also acknowledges a key real-world challenge: many images may contain background clutter or no mango at all. For this reason, the author added negative-detection handling so the system can return `No mango detected` rather than forcing a wrong class.

# 3 Primary Source of This Report

The primary source for this report is the saved notebook artifact `advance_mango_detection.ipynb` found in the project workspace. All implementation details, variable names, thresholds, pipeline steps, model choices, and execution-status observations in this report were extracted directly from that notebook. External references are used only to identify the official sources of the datasets, frameworks, and model families cited by the notebook.

# 4 Notebook Audit Summary

## 4.1 Structure

The saved notebook contains **36 cells** and is organized into **15 named sections**, beginning with environment setup and ending with a Streamlit deployment cell.

## 4.2 Execution state

The notebook metadata shows it was developed in **Google Colab** with a **Tesla T4 GPU**. However, the saved execution state is partial:

- the package installation, dataset download, detector training, and detector inference cells were executed;
- the later ripeness-classification, end-to-end test, model export, and Streamlit-writing cells are present in code but were **not executed** in the saved notebook.

This matters for honest reporting: the notebook contains a complete intended pipeline, but the saved notebook only records direct evidence for the **detection stage**, not the full trained classification stage.

## 4.3 Section-by-section audit

| Step | Notebook Section | Purpose |
|---|---|---|
| 1 | Install libraries | Installs `ultralytics`, `roboflow`, `tensorflow`, `streamlit`, `opencv-python`, and `kaggle`. |
| 2 | Import libraries and settings | Defines paths, thresholds, class names, model file locations, and global constants. |
| 3 | Inference utilities | Implements reusable functions for detection, cropping, preprocessing, classification, and full image processing. |
| 4 | Roboflow dataset download | Downloads the detection dataset in YOLOv8 format. |
| 5 | Detector training | Fine-tunes `yolov8n.pt` for mango detection. |
| 6 | Test inference | Runs the trained detector on test images and saves visualizations. |
| 7 | Google Drive backup | Mounts Drive and copies training outputs for persistence in Colab. |
| 8 | Best-weight reload and review | Reloads best detector weights and previews outputs. |
| 9 | Crop generation | Uses the trained detector to create mango crops for downstream use. |
| 10 | Kaggle dataset download | Downloads the ripeness dataset archive from Kaggle. |
| 11 | TensorFlow data loading | Creates train and validation generators with a validation split. |
| 12 | Classifier construction and training | Builds an EfficientNetB0 transfer-learning classifier and trains it. |
| 13 | Full pipeline test | Runs detection plus ripeness prediction on a sample image. |
| 14 | Model export | Saves the detector weights and the classifier model. |
| 15 | Streamlit app generation | Writes an application script for interactive inference. |

# 5 System Objective and Functional Scope

The notebook aims to build an end-to-end mango-analysis system with the following functional capabilities:

- detect one or more mangoes in an input image,
- crop each detected mango,
- classify ripeness for each crop,
- reject low-confidence classification results using an `uncertain` label,

- report when no mango is detected,
- annotate the input image with boxes and confidence information,
- export models for later use,
- provide a deployable Streamlit interface.

# 6 Technical Architecture

## 6.1 Two-stage pipeline

The notebook uses a cascaded architecture:

1. **Input image acquisition**
2. **YOLOv8 detection** to localize mangoes with bounding boxes
3. **Crop extraction** with a small margin around each box
4. **Image preprocessing** for the classifier
5. **EfficientNetB0 classification** of each crop
6. **Confidence filtering** to suppress uncertain ripeness predictions
7. **Annotation and summary generation**

## 6.2 Why this architecture is appropriate

This design is appropriate because:

- detection and ripeness classification are different tasks,
- object detection naturally handles multiple mangoes in one image,
- crop-level classification is more focused than classifying the entire scene,
- confidence-aware filtering improves practical reliability.

## 6.3 Confidence logic

The notebook computes two forms of confidence:

- **detection confidence**, returned by YOLO;
- **classification confidence**, returned by the softmax output of the classifier.

It also computes a combined score:

$$\text{combined confidence} = \text{detection confidence} \times \text{classification confidence}$$

This combined score is used as a compact indicator of end-to-end confidence for each mango.

# 7 Detailed Implementation Analysis

## 7.1 Imported libraries

The notebook imports:

- `os` and `pathlib.Path` for filesystem handling,
- `typing` for type hints,
- `cv2` for image I/O and preprocessing,
- `numpy` for tensor and array operations,
- `tensorflow` for classifier definition and training,
- `IPython.display` for in-notebook visualization,
- `roboflow` for dataset download,
- `tensorflow.keras.preprocessing` utilities for image loading and generators,
- `ultralytics.YOLO` for detection training and inference.

## 7.2 Global settings and constants

The project-level constants extracted from the notebook are shown in Table 2.

Table 2: Key constants defined in the notebook

| Constant | Value / Meaning |
|---|---|
| WORKSPACE_NAME | pigrecipe |
| PROJECT_NAME | mango-7l7ug-napwv-razzf |
| PROJECT_VERSION | 1 |
| DATASET_DIR | mango-1 |
| YOLO_DATA_CONFIG | mango-1/data.yaml |
| YOLO_BASE_WEIGHTS | yolov8n.pt |
| YOLO_RUNS_DIR | /content/runs/detect |
| YOLO_BEST_WEIGHTS | /content/runs/detect/train/weights/best.pt |
| PREDICT_DIR | /content/runs/detect/predict |
| CROP_OUTPUT_DIR | /content/mango_crops |
| RIPENESS_DATA_DIR | /content/ripeness_data |
| CLASSIFIER_MODEL_PATH | /content/mango_classifier.h5 |
| CLASS_NAMES | {raw, ripe, overripe} |
| DETECTION_CONF_THRESH | 0.5 |
| CLASSIFICATION_CONF_THRESH | 0.6 |
| CROP_MARGIN | 10 pixels |
| CLASSIFIER_IMAGE_SIZE | (224, 224) |

## 7.3 Inference utility functions

The notebook includes five core helper functions:

1. `detect_mangoes`: calls YOLO prediction, extracts bounding boxes and confidences, and keeps only boxes above the detection threshold;
2. `crop_mango`: enlarges the box by a fixed margin and crops the fruit region;
3. `preprocess_crop`: resizes the crop to $224 \times 224$, scales pixel values to $[0, 1]$, and adds a batch dimension;
4. `classify_mango`: returns the ripeness label with softmax confidence, or `uncertain` if confidence is below 0.6;
5. `process_image`: performs full end-to-end inference and generates a structured summary dictionary.

These utilities are the most important engineering contribution of the notebook because they convert separate model components into a reusable inference pipeline.

# 8 Datasets and Data Sources

## 8.1 Detection dataset

The detector training data is downloaded from Roboflow using the Python SDK and exported in YOLOv8 format [3, 4]. The notebook records:

- workspace: `pigrecipe`,
- project: `mango-7l7ug-napwv-razzf`,
- version: `1`,
- local extracted location: `/content/mango-1`.

The saved notebook output also confirms that the extracted dataset contains:

- `train`,
- `valid`,
- `test`,
- `data.yaml`,
- Roboflow README files.

During prediction, the model reports a single detection class named `Fresh Mango`. This indicates that the detector is trained as a one-class mango localizer, not as a ripeness classifier.

## 8.2 Ripeness classification dataset

The second dataset is downloaded from Kaggle using [5]:

```
kaggle datasets download -d srabon00/mango-ripening-stage-classification
```

It is extracted into `/content/ripeness_data`. The notebook uses directory-based loading, meaning the ripeness classes are expected to be organized into class-named subfolders. The project-level class names are explicitly defined as:

`raw`, `ripe`, `overripe`

## 8.3 Why two datasets are used

The notebook intentionally separates localization data from ripeness data:

- the Roboflow dataset teaches the model where mangoes are;
- the Kaggle dataset teaches the model how ripeness appearances differ.

This separation is reasonable when a single dataset does not provide both bounding boxes and ripeness labels at object level.

# 9 Training Configuration

## 9.1 YOLOv8 detector

The detector configuration recorded in the notebook is [2]:

- base model: `yolov8n.pt`,
- training epochs: `50`,
- image size: `640`,
- batch size: `16`,
- platform: Google Colab with Tesla T4 GPU,
- library family: Ultralytics YOLOv8.

The choice of `yolov8n` suggests a speed-oriented detector suitable for demonstration or lightweight deployment.

## 9.2 EfficientNetB0 classifier

The classification stage uses transfer learning [6, 7, 8]:

- base network: `EfficientNetB0`,
- pretrained weights: `imagenet`,
- input size: $224 \times 224 \times 3$,
- top removed: `include_top=False`,
- backbone frozen for the initial pass,
- added head: global average pooling $\rightarrow$ dense(128, ReLU) $\rightarrow$ dense(3, softmax),
- optimizer: Adam,
- loss: categorical cross-entropy,
- metric: accuracy,
- training epochs: `10`.

### 9.3 Data loading for the classifier

TensorFlow's `ImageDataGenerator` is used with [9]:

- pixel rescaling by $\frac{1}{255}$,
- validation split of `0.2`,
- target size $224 \times 224$,
- batch size `32`.

This creates a straightforward supervised learning setup with directory-based labels.

## 10 Execution Evidence from the Saved Notebook

### 10.1 Observed completed stages

The notebook outputs confirm that the following stages were executed:

- package installation,
- Roboflow dataset download,
- detector training invocation,
- detector inference on the test split,
- listing of dataset directories.

### 10.2 Observed detector evidence

The saved output provides the following direct evidence:

- the dataset was downloaded to `/content/mango-1`;
- the test split used by prediction contains **50 images**, as indicated by the output pattern `image 1/50`;
- prediction logs show detections such as `1 Fresh Mango`, `2 Fresh Mangos`, `6 Fresh Mangos`;
- many inference times shown in the saved output are around **5.8 ms** per image, with some larger values for different resolutions.

### 10.3 Observed incomplete stages

The following code exists but was not executed in the saved notebook:

- reloading the best weights for clean review,
- displaying `results.png`,
- visual preview of saved prediction images,
- crop generation,
- Kaggle dataset extraction and classifier training,
- full-pipeline sample test,
- classifier prediction on a crop,
- export of the classifier model,
- writing of `app.py`.

Therefore, the notebook demonstrates **implementation completeness in code** but only **partial experimental completeness in saved outputs**.

## 11 Step-by-Step Operational Flow

The notebook's end-to-end workflow can be described as follows:

1. Install all required Python packages in a fresh Colab environment.
2. Import libraries and define paths, thresholds, classes, and output locations.
3. Define reusable functions for detection, cropping, preprocessing, classification, and full-pipeline inference.
4. Download a YOLO-formatted mango detection dataset from Roboflow.
5. Inspect the dataset structure to verify expected folders.
6. Fine-tune YOLOv8n on the detection dataset.
7. Run detector inference on the test split and save visualized predictions.
8. Optionally copy training artifacts to Google Drive for persistence.
9. Reload the best trained detector weights for cleaner evaluation.
10. Use the detector to generate mango crops from training images.
11. Download a ripeness classification dataset from Kaggle.

12. Build TensorFlow training and validation generators.
13. Create an EfficientNetB0-based classifier and train it.
14. Run the full detection-plus-ripeness pipeline on a test image.
15. Save the best detector and trained classifier for later deployment.
16. Generate a Streamlit app that exposes the same inference logic in a user interface.

# 12 The Streamlit Deployment Layer

The final notebook cell writes a Streamlit application named `app.py` [10]. The application:

- loads the YOLO detector and Keras classifier once per session using `st.cache_resource`,
- accepts image uploads in `jpg`, `jpeg`, or `png` format,
- decodes the image with OpenCV,
- displays the uploaded image,
- runs the full detection-plus-classification pipeline,
- returns an error if no mango is detected,
- otherwise displays the annotated image, mango counts, and average confidence values.

This deployment step is significant because it converts the notebook from an experimental workflow into a usable application prototype.

# 13 Design Strengths

The notebook shows several good engineering decisions:

- **task decomposition**: detection and ripeness classification are handled separately;
- **reusable helpers**: inference logic is not duplicated across cells;
- **confidence thresholds**: helps reduce low-confidence errors;
- **uncertain label**: avoids forcing fragile predictions;
- **no-mango handling**: improves real-world behavior;
- **margin-based cropping**: reduces overly tight crop errors;
- **deployment awareness**: the notebook includes export and app code, not only training code.

# 14 Limitations and Risks

## 14.1 Partial execution record

The biggest reporting limitation is that the saved notebook does not include executed results for the classification stage. As a result, no classifier accuracy, confusion matrix, precision, recall, F1-score, or final end-to-end evaluation can be truthfully claimed from the saved artifact alone.

## 14.2 Hard-coded secret

The notebook contains a Roboflow API key directly in code. This is a security risk and should be replaced by an environment variable or secret-management workflow.

## 14.3 Cross-dataset domain gap

The pipeline combines a detection dataset from Roboflow and a ripeness dataset from Kaggle. This can introduce a domain shift because the lighting conditions, backgrounds, camera angles, and fruit presentation styles may differ between datasets.

## 14.4 Unclear label alignment

The detector output class shown in logs is `Fresh Mango`, while the classifier classes are `raw`, `ripe`, and `overripe`. This is not an error by itself, but it shows that detection and ripeness semantics come from different labeling systems and should be documented carefully.

## 14.5 Class imbalance risk

The ripeness classes may be imbalanced, which could reduce classifier reliability for minority ripeness stages unless augmentation, balancing, or weighted loss is added.

# 15 Improvement Plan Documented by the Notebook

The notebook already includes a useful negative-detection improvement plan. It recommends:

- adding background-only images with no mango annotations,
- including hard negatives such as other fruits, leaves, baskets, branches, and indoor objects,
- leaving such images unlabeled so YOLO learns not to predict boxes,

- re-evaluating precision on a mixed validation set containing mango and non-mango scenes.

This is a strong recommendation because reliable `No mango detected` behavior requires negative examples during training.

# 16 Recommendations for a More Complete Final Project

To convert this notebook into a stronger final-year or professional portfolio project, the following additions are recommended:

1. fully execute the ripeness-training and end-to-end evaluation cells;
2. record classifier metrics such as accuracy, precision, recall, and confusion matrix;
3. report detector metrics such as precision, recall, mAP@0.5, and mAP@0.5:0.95 from the saved training outputs;
4. save key figures such as prediction samples and training curves into the repository;
5. replace the hard-coded API key with secure secrets handling;
6. add a reproducibility note covering environment versions and dataset versions;
7. evaluate the deployed Streamlit app on both positive and negative test images.

# 17 Conclusion

The notebook implements a sensible and practically motivated two-stage mango-analysis system. It is designed to answer the question: *where are the mangoes in an image, and what is the ripeness state of each detected mango?* It does so by combining YOLOv8 object detection, crop extraction, EfficientNetB0 transfer learning, threshold-based decision rules, and Streamlit deployment. The strongest parts of the notebook are its modular inference utilities, its confidence-aware logic, and its end-to-end application mindset. The main gap is not in the design, but in the saved execution record: the detector stage was run, while the classifier and application stages remain defined in code but unexecuted in the saved notebook. Even with that limitation, the notebook represents a coherent foundation for an agricultural computer vision project and can be presented professionally once the missing execution evidence and evaluation metrics are added.

# 18 References

## References


[1] Project notebook under analysis. `advance_mango_detection.ipynb`, local workspace artifact, accessed 23 April 2026.

[2] Ultralytics. *Ultralytics YOLO Documentation*. Available at: https://docs.ultralytics.com/

[3] Roboflow. *Roboflow Python Package and Dataset Workflow Documentation*. Available at: https://docs.roboflow.com/developer/export-data

[4] Roboflow Universe. *Mango Detection Dataset: project* ***mango-7l7ug-napwv-razzf***. Available at: https://universe.roboflow.com/

[5] Kaggle. *Mango Ripening Stage Classification Dataset by* ***srabon00***. Available at: https://www.kaggle.com/datasets/srabon00/mango-ripening-stage-classification

[6] Keras. *EfficientNetB0 and Keras Applications API*. Available at: https://keras.io/api/applications/efficientnet/

[7] Mingxing Tan and Quoc V. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. Available at: https://arxiv.org/abs/1905.11946

[8] TensorFlow. *Transfer Learning and Fine-Tuning Guide*. Available at: https://www.tensorflow.org/tutorials/images/transfer_learning

[9] TensorFlow / Keras. *ImageDataGenerator API Reference*. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator

[10] Streamlit. *Streamlit Documentation*. Available at: https://docs.streamlit.io/